Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Manejo e Implementacion de Archivos

Sección: A

Docente: Bryan Misael Monzon Fuentes

“Manual Tecnico Proyecto 1”

Estudiante:

Mynor Miguel Monzón Martínez 202230884

Quetzaltenango, 22 de septiembre de

2026

Introducción

El presente manual técnico documenta el diseño y la implementación del Mini ERP, un sistema de gestión comercial desarrollado para una empresa dedicada a la venta y distribución de muebles para el hogar y la oficina. El documento está dirigido al personal técnico encargado de revisar, mantener o ampliar el sistema, y complementa al Manual de Usuario, que describe el uso funcional de la aplicación.

El sistema integra la administración de productos, categorías, proveedores, clientes, compras, ventas e inventario, aplicando control de acceso por roles, control de existencias mediante el método PEPS (primeras entradas, primeras salidas), generación de facturas en PDF y un módulo de reportes con fines de análisis y auditoría.

1. Arquitectura del Sistema

1.1 Arquitectura cliente-servidor

El sistema está compuesto por dos aplicaciones independientes que se comunican mediante una API REST sobre HTTP:

- **Aplicación cliente:** desarrollada en Angular 21, se ejecuta en el navegador del usuario y consume la API mediante peticiones HTTP con formato JSON.
- **Aplicación servidor:** desarrollada en Spring Boot con Java 21, expone la API REST y es la única responsable de comunicarse con la base de datos. El cliente nunca accede a la base de datos de forma directa.

La base de datos PostgreSQL es de uso exclusivo del servidor. Esta separación permite que ambas aplicaciones evolucionen y se desplieguen de forma independiente.

1.2 Arquitectura en capas del backend

Dentro del servidor, el código se organiza en capas con responsabilidades separadas, siguiendo el patrón habitual de las aplicaciones Spring Boot:

Capa	Paquete	Responsabilidad
Presentación	controller	Expone los endpoints REST, recibe y valida las peticiones HTTP y devuelve las respuestas.
Transferencia de datos	dto	Define la forma de los datos que entran y salen de la API, independiente de las entidades.
Negocio	service	Implementa las reglas de negocio, coordina las transacciones y orquesta los repositorios.
Acceso a datos	repository	Interfaces de Spring Data JPA que consultan y persisten información en la base de datos.
Modelo	entity	Clases anotadas con JPA que representan las tablas de la base de datos.
Seguridad	security	Autenticación y autorización mediante JWT.

Capa	Paquete	Responsabilidad
Configuración	config	Configuración de seguridad, CORS y codificación de contraseñas.
Manejo de errores	exception	Excepciones propias del negocio y su traducción a respuestas HTTP.

Una petición típica recorre estas capas de la siguiente manera: el **controller** recibe la petición y delega en un **service**; el **service** aplica las reglas de negocio y utiliza uno o varios **repository** para leer o escribir en la base de datos a través de las **entity**; la respuesta se transforma a un **dto** antes de regresar al cliente. El controlador no contiene lógica de negocio, el servicio no conoce detalles de HTTP y el repositorio no conoce reglas de negocio.

2. Tecnologías y Herramientas

Componente	Tecnología	Versión
Lenguaje del servidor	Java	21
Framework del servidor	Spring Boot	4.1.1
Persistencia	Spring Data JPA / Hibernate	7.4.5
Migraciones de base de datos	Flyway	12.4.0
Motor de base de datos	PostgreSQL	16
Autenticación	Spring Security + JWT (jjwt)	0.12.6
Generación de PDF	OpenPDF	2.0.3
Reducción de código repetitivo	Lombok	1.18.42
Gestor de dependencias del servidor	Maven	—
Framework del cliente	Angular	21
Lenguaje del cliente	TypeScript	—
Entorno de ejecución del cliente	Node.js	22
Control de versiones	Git / GitHub	—

Como herramientas de desarrollo se utilizaron IntelliJ IDEA para el backend, Visual Studio Code para el frontend, DBeaver como cliente gráfico de PostgreSQL y draw.io para los diagramas de este proyecto.

3. Estructura del Repositorio

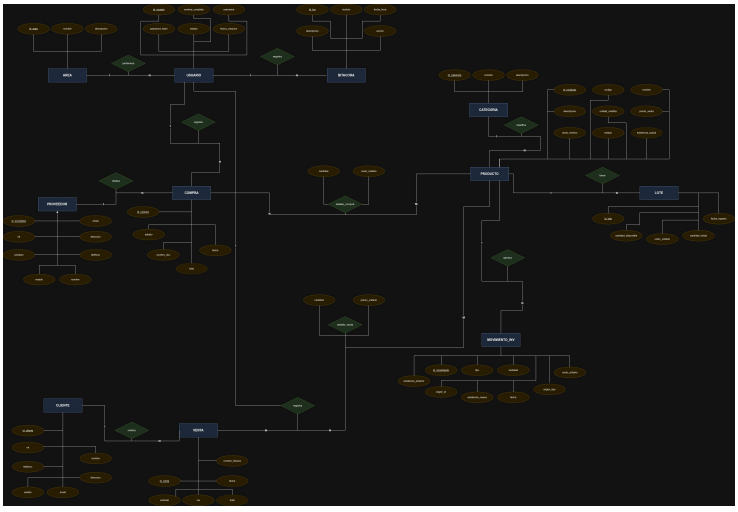
El repositorio se organiza en tres carpetas principales, correspondientes a las dos aplicaciones y a la documentación:

```
Proyecto_1_Manejo_De_Archivos_Mynor_Monzon_2S_2026/
├── backend/                    Aplicacion servidor (Spring Boot)
│   └── src/main/
│       ├── java/com/minierp/backend/
│       │   ├── config/         Configuracion de seguridad y CORS
│       │   ├── controller/     Controladores REST
│       │   ├── dto/            Objetos de transferencia de datos
│       │   ├── entity/         Entidades JPA
│       │   ├── exception/      Excepciones y manejo global de errores
│       │   ├── repository/     Repositorios Spring Data JPA
│       │   ├── security/       JWT, filtro y detalle de usuario
│       │   └── service/        Reglas de negocio
│       └── resources/
│           ├── application.yml
│           └── db/migration/ Scripts Flyway (V1, V2)
├── frontend/                   Aplicacion cliente (Angular)
│   └── src/app/
│       ├── core/               Modelos, servicios, guards e interceptor
│       ├── layout/             Menu lateral y barra superior
│       └── pages/              Una carpeta por pantalla
└── docs/                       Diagramas y manuales del proyecto
```

4. Modelo de Datos

4.1 Diagrama entidad-relación

El modelo de datos se diseñó utilizando notación de Peter Chen, identificando las entidades, sus atributos y las relaciones entre ellas antes de transformarlo en el modelo relacional. El diagrama completo se encuentra en el archivo `docs/ER_MiniERP.png`



4.2 Diccionario de datos

La base de datos está compuesta por trece tablas. A continuación se describe el propósito de cada una y sus llaves foráneas:

Tabla	Descripción	Llaves foráneas
area	Áreas de la empresa (Administración, Compras, Inventario, Ventas). Determina el rol de cada usuario.	—
usuario	Empleados con acceso al sistema. La contraseña se almacena como hash BCrypt.	id_area → area
categoria	Clasificación de los productos (escritorios, sillas, mesas, etc.).	—
producto	Catálogo de muebles. La existencia_actual es un valor derivado que solo cambia por compras y ventas.	id_categoria → categoria
proveedor	Empresas que abastecen los productos. Borrado lógico mediante el campo estado.	—
cliente	Personas o empresas a las que se vende. Borrado lógico mediante el campo estado.	—
compra	Encabezado de cada adquisición realizada a un proveedor.	id_proveedor → proveedor, id_usuario → usuario
detalle_compra	Líneas de producto de una compra. Representa la relación M:N entre compra y producto.	id_compra → compra, id_producto → producto
lote	Capa de inventario creada por cada compra, con su costo y cantidad disponible. Es la base del método PEPS.	id_producto → producto, id_detalle_compra → detalle_compra
venta	Encabezado de cada venta realizada a un cliente, con subtotal, IVA y total.	id_cliente → cliente, id_usuario → usuario
detalle_venta	Líneas de producto de una venta. Representa la relación M:N entre venta y producto.	id_venta → venta, id_producto → producto
movimiento_inv	Historial de entradas y salidas de inventario, con su origen, costo y existencia antes y después.	id_producto → producto, id_usuario → usuario
bitacora	Registro de las acciones realizadas por los usuarios en el sistema.	id_usuario → usuario

4.3 Migraciones con Flyway

El enunciado exige que la base de datos se cree y cargue mediante scripts SQL versionados. Por esta razón, la estructura y los datos nunca se crean con Hibernate: se crean exclusivamente con **Flyway**, a partir de dos migraciones:

- **V1__crear_tablas.sql:** crea las trece tablas, sus llaves primarias y foráneas, restricciones `CHECK` (existencias y cantidades no negativas, montos positivos) e índices para los reportes.
- **V2__datos_iniciales.sql:** inserta la carga inicial mínima exigida (4 usuarios, 30 productos, 8 proveedores, 20 clientes, 15 compras y 25 ventas), generada de forma que respeta las mismas reglas de negocio que el sistema: cada compra crea sus lotes y movimientos, y cada venta consume lotes por PEPS.

Al iniciar, Spring Boot ejecuta Flyway antes de que Hibernate valide el modelo. En `application.yml` la propiedad `spring.jpa.hibernate.ddl-auto` está configurada como `validate`, de modo que Hibernate únicamente compara las entidades contra las tablas existentes y nunca las crea ni las modifica.

```
spring:
  jpa:
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true
    locations: classpath:db/migration
```

5. Seguridad

5.1 Autenticación con JWT

El sistema utiliza JSON Web Tokens (JWT) y no mantiene sesiones en el servidor (`SessionCreationPolicy.STATELESS`). El flujo es el siguiente:

1. El usuario envía su `username` y `password` a `POST /api/auth/login`.
2. `AuthService` delega en el `AuthenticationManager` de Spring Security, que compara la contraseña recibida contra el hash BCrypt almacenado en la tabla `usuario`.
3. Si las credenciales son correctas, `JwtService` genera un token firmado con una clave secreta, con el nombre de usuario, el área y una expiración de 8 horas.
4. El cliente guarda el token y lo envía en cada petición posterior en el encabezado `Authorization: Bearer <token>`.
5. `JwtAuthFilter`, ejecutado una vez por petición, valida la firma del token y registra al usuario como autenticado ante Spring Security.

5.2 Autorización por roles

El área de cada usuario (`Administración`, `Compras`, `Inventario`, `Ventas`) se traduce en un rol de Spring Security con el prefijo `ROLE_`. `SecurityConfig` declara, para cada ruta de la API, qué roles pueden acceder. El área de Administración tiene acceso a todos los módulos; las demás áreas

únicamente a los que les corresponden. Una petición sin token recibe **401 Unauthorized**; una petición con un rol sin permiso recibe **403 Forbidden**.

5.3 Cifrado de contraseñas

Las contraseñas nunca se almacenan en texto plano. `BCryptPasswordEncoder` genera un hash con sal aleatoria y factor de costo 10 al crear un usuario, y el mismo mecanismo se usa para verificar la contraseña en cada inicio de sesión, sin que el sistema necesite conocer ni revertir la contraseña original.

6. Lógica de Negocio

6.1 Control de existencias

La columna `existencia_actual` de `producto` es un valor derivado: no puede modificarse manualmente desde la API. Únicamente cambia como consecuencia de una compra (entrada) o una venta (salida), y cada cambio se realiza dentro de `InventarioService`, que también dejar registrado un `movimiento_inv` con el origen exacto de la operación. Antes de completar una venta, el sistema valida que exista existencia suficiente; de lo contrario, la operación se rechaza con **409 Conflict** y no se modifica el inventario.

6.2 Método PEPS

El enunciado exige aplicar UEPS o PEPS según la paridad del carnet del estudiante; a este proyecto corresponde **PEPS** (primeras entradas, primeras salidas). Cada compra crea un `lote` con su propio costo y fecha de ingreso. Al vender, `InventarioService.registrarSalidaPeps` obtiene los lotes del producto ordenados por fecha de ingreso ascendente y va descontando la cantidad vendida del lote más antiguo con existencia disponible, pasando al siguiente si no alcanza:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT l FROM Lote l WHERE l.producto.idProducto = :idProducto",
     + " AND l.cantidadDisponible > 0 ORDER BY l.fechaIngreso ASC,
l.idLote ASC")
List<Lote> findDisponiblesPeps(@Param("idProducto") Integer idProducto);
```

Como una venta puede consumir varios lotes con costos distintos, se genera un movimiento de salida por cada lote consumido, cada uno con el costo real de ese lote. El costo unitario que se guarda en el detalle de venta es el promedio ponderado de los lotes consumidos, lo que permite calcular la utilidad real de cada venta.

El bloqueo `PESSIMISTIC_WRITE` evita que dos ventas simultáneas consuman las mismas unidades de un lote: la segunda transacción espera a que la primera libere el bloqueo antes de leer la existencia disponible.

6.3 Transaccionalidad

Cada operación que involucra más de una escritura (`CompraService.registrar`, `VentaService.registrar`) está anotada con `@Transactional`. Si en medio de la operación se lanza una excepción, por ejemplo `NegocioException` por falta de existencias, Spring ejecuta un `rollback` automático: ninguna de las escrituras parciales (compra, detalle, lote, movimiento, o

venta, detalle, movimiento) queda guardada. Esto garantiza que el inventario nunca quede en un estado inconsistente.

6.4 Generación de la factura en PDF

Al registrar una venta, o al consultarla posteriormente, `FacturaPdfService` genera un documento PDF con la librería OpenPDF, incluyendo los datos del cliente, el detalle de productos, el subtotal, el IVA (12 %) y el total. El documento se devuelve directamente como flujo de bytes en la respuesta HTTP (`GET /api/ventas/{id}/factura`), sin almacenarse en el servidor.

6.5 Bitácora de auditoría

`BitacoraService.registrar` se invoca dentro de la misma transacción de cada operación relevante (creación y actualización de catálogos, cambios de estado, registro de compras y ventas). Si la operación falla y se revierte, el registro de bitácora se revierte con ella, de modo que la bitácora nunca refleja una acción que en realidad no se completó.

7. API REST

Todos los endpoints, salvo el de inicio de sesión, requieren el encabezado `Authorization: Bearer <token>`. La columna Roles indica, además de Administración (que siempre tiene acceso), qué otras áreas pueden usar el endpoint.

7.1 Autenticación

Método	Ruta	Roles	Descripción
POST	/api/auth/login	Público	Autentica al usuario y devuelve el token JWT

7.2 Categorías y productos

Método	Ruta	Roles	Descripción
GET	/api/categorias	Todos	Lista las categorías
POST	/api/categorias	Inventario	Crea una categoría
PUT	/api/categorias/{id}	Inventario	Actualiza una categoría
GET	/api/productos	Todos	Lista los productos
GET	/api/productos/stock-bajo	Todos	Productos en o bajo el stock mínimo
GET	/api/productos/{id}	Todos	Obtiene un producto
POST	/api/productos	Inventario	Crea un producto (existencia inicial 0)
PUT	/api/productos/{id}	Inventario	Actualiza un producto
PATCH	/api/productos/{id}/estado	Inventario	Activa o desactiva (borrado lógico)

7.3 Proveedores, clientes y usuarios

Método	Ruta	Roles	Descripción
GET/POST/PUT	/api/proveedores	Compras	CRUD de proveedores
PATCH	/api/proveedores/{id}/estado	Compras	Activa o desactiva
GET/POST/PUT	/api/clientes	Ventas	CRUD de clientes
PATCH	/api/clientes/{id}/estado	Ventas	Activa o desactiva
GET	/api/usuarios	Solo Admin.	Lista los usuarios
GET	/api/usuarios/areas	Solo Admin.	Lista las áreas
POST	/api/usuarios	Solo Admin.	Crea un usuario
PATCH	/api/usuarios/{id}/estado	Solo Admin.	Activa o desactiva

7.4 Compras, ventas e inventario

Método	Ruta	Roles	Descripción
GET	/api/compras?desde&hasta	Compras	Lista compras por rango de fechas
GET	/api/compras/{id}	Compras	Obtiene una compra con su detalle
POST	/api/compras	Compras	Registra una compra (crea lotes y movimientos)
GET	/api/ventas?desde&hasta&idCliente	Ventas	Lista ventas, con filtro opcional por cliente
GET	/api/ventas/{id}	Ventas	Obtiene una venta con su detalle
POST	/api/ventas	Ventas	Registra una venta (valida stock y aplica PEPS)
GET	/api/ventas/{id}/factura	Ventas	Descarga la factura en PDF
GET	/api/inventario/productos/{id}/movimientos	Inventario	Historial de movimientos de un producto
GET	/api/inventario/productos/{id}/lotes	Inventario	Lotes disponibles de un producto

7.5 Reportes y bitácora

Ruta	Roles	Descripción
/api/reportes/inventario/mas-vendidos	Inventario	Top productos más vendidos
/api/reportes/inventario/menor-existencia	Inventario	Productos con menor existencia
/api/reportes/inventario/mas-movimientos	Inventario	Productos con más movimientos
/api/reportes/inventario/historial/{idProducto}	Inventario	Historial de un producto
/api/reportes/compras/rango	Compras	Compras por rango de fechas
/api/reportes/compras/top-proveedores	Compras	Top 5 proveedores por monto
/api/reportes/compras/mas-adquiridos	Compras	Productos más adquiridos
/api/reportes/ventas/rango	Ventas	Ventas por rango de fechas
/api/reportes/ventas/top-clientes	Ventas	Top 10 clientes por monto
/api/reportes/ventas/top-ingresos	Ventas	Top 10 productos por ingresos
/api/reportes/ventas/resumen	Ventas	Resumen de ventas por período
/api/reportes/logs	Solo Admin.	Reporte de logs (bitácora)
/api/bitacora	Solo Admin.	Bitácora de operaciones

Todos los reportes son de tipo `GET` y aceptan los parámetros opcionales que correspondan: `limite`, `desde`, `hasta` o `agrupacion` (día, semana, mes o año).

8. Frontend

8.1 Arquitectura

La aplicación cliente utiliza componentes **standalone** de Angular 21, sin `NgModule`, y el estado de cada componente se maneja con **signals** en lugar de `zone.js`. La comunicación con la API se realiza mediante `HttpClient` y `Observables` de RxJS.

8.2 Carpeta core

Subcarpeta	Contenido
models	Interfaces de TypeScript que reflejan la forma de los DTOs del backend.
services	Un servicio por módulo (productos, compras, ventas, reportes, etc.) que realiza las peticiones HTTP.
interceptors	Agrega el token a cada petición y cierra la sesión si el backend responde 401.
guards	Impiden el acceso a una ruta sin sesión iniciada o sin permiso del área correspondiente.

8.3 Layout y pages

`layout` contiene el menú lateral y la barra superior que envuelven a todas las pantallas; el menú se filtra dinámicamente según el área de la sesión activa. `pages` contiene una carpeta por pantalla (14 en total: login, inicio, categorías, productos, proveedores, clientes, usuarios, compras, compra-nueva, ventas, venta-nueva, inventario, reportes y bitácora), cada una con sus archivos `.ts`, `.html` y `.css`.

8.4 Seguridad en el cliente

El token JWT se guarda en `localStorage` y el interceptor lo agrega automáticamente a cada petición. Las rutas están protegidas por `authGuard` (sesión iniciada) y `rolGuard` (permiso del área). Es importante aclarar que estas protecciones son una comodidad de navegación para el usuario: la seguridad real reside en el backend, que rechaza con 401 o 403 cualquier petición no autorizada, incluso si alguien intentara acceder directamente a la API sin pasar por la interfaz.

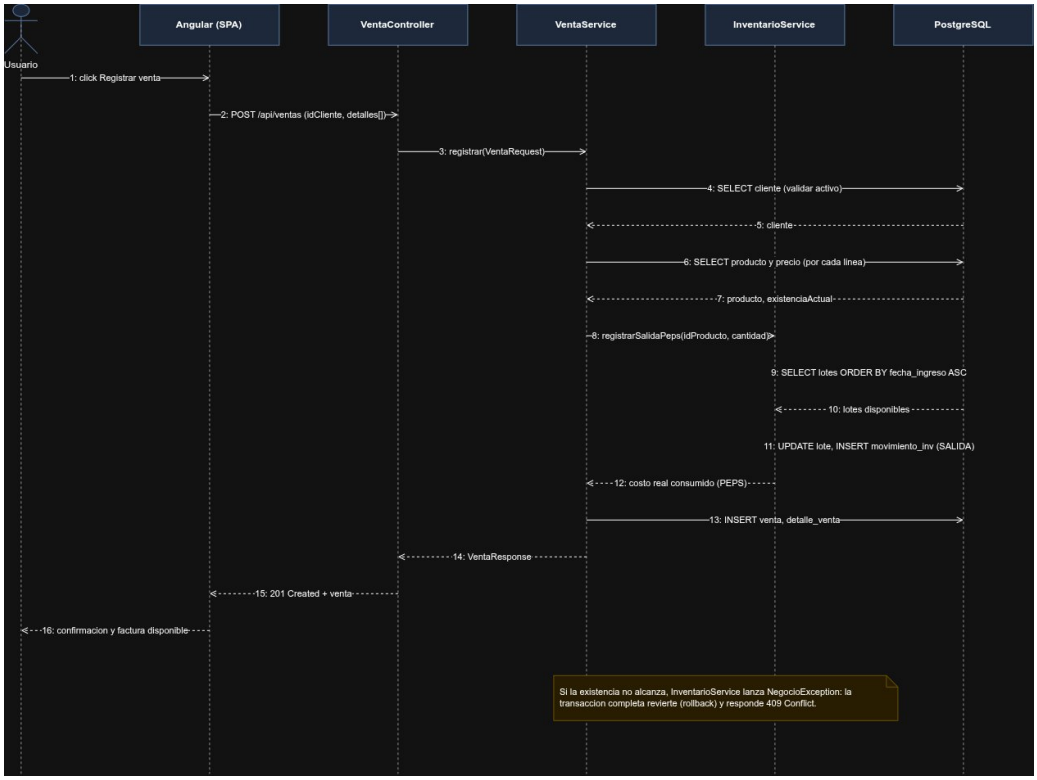
9. Diagramas UML

Los siguientes diagramas complementan este manual y se encuentran en la carpeta `docs/` del repositorio, elaborados en draw.io.

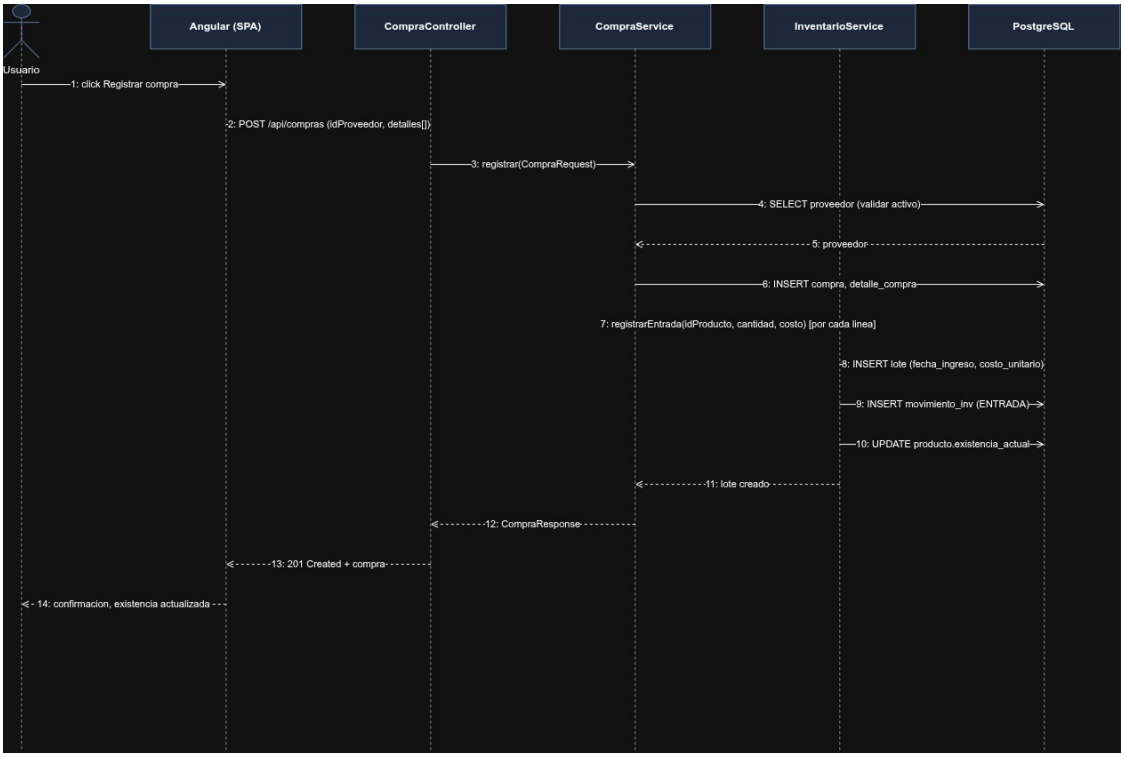
9.1 Diagrama de casos de uso



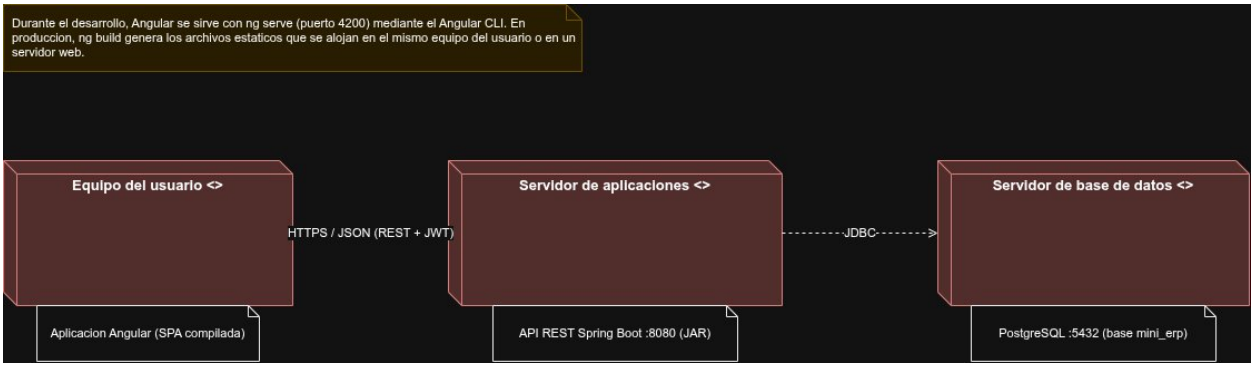
9.2 Diagrama de secuencia: registrar una venta



9.3 Diagrama de secuencia: registrar una compra



9.4 Diagrama de despliegue



10. Pruebas Realizadas

Durante el desarrollo se realizaron pruebas manuales sobre los endpoints y sobre la interfaz, verificando en particular:

- Inicio de sesión correcto e incorrecto, y ausencia de token (401).
- Restricción de acceso por rol: un usuario del área de Ventas recibe 403 al intentar consultar `/api/proveedores`.
- Aplicación de PEPS: dos compras del mismo producto a costos distintos, seguidas de una venta que consume ambos lotes en el orden correcto y con el costo real de cada uno.
- Rechazo de una venta por stock insuficiente (409), verificando que el inventario no se modificó.
- Consistencia del inventario: la existencia actual de un producto coincide con la suma de sus lotes disponibles y con la existencia nueva de su último movimiento.
- Generación correcta de la factura en PDF, con los montos de subtotal, IVA y total.
- Reportes de resumen de ventas por período y de proveedores con mayor monto de compra.
- Flujo completo desde la interfaz: nueva compra, nueva venta y verificación del inventario resultante.

11. Decisiones Técnicas y Justificación

Decisión	Justificación
`NUMERIC` en lugar de `FLOAT`/`DOUBLE` para montos	Evita errores de redondeo binario en valores monetarios, que serían inaceptables en compras y ventas.
`ddl-auto: validate` en lugar de `update` o `create`	La base de datos debe crearse únicamente mediante los scripts Flyway del estudiante; Hibernate solo valida que las entidades coincidan con las tablas.
Borrado lógico (columna `estado`) en lugar de `DELETE`	El enunciado exige conservar el historial de compras, ventas y movimientos aunque un elemento deje de utilizarse.
DTOs en lugar de exponer las entidades	Evita filtrar datos sensibles (como el hash de la contraseña), permite validar los datos de entrada y desacopla la API del modelo de base

Decisión	Justificación
	de datos.
`@Lock(PESSIMISTIC_WRITE)` sobre los lotes	Evita que dos ventas simultáneas consuman las mismas unidades de inventario, lo que podría producir existencias negativas.
JWT sin sesiones en el servidor	Simplifica el escalamiento de la API y es el estándar para APIs REST consumidas por una SPA.
Tabla `lote` independiente de `producto`	Sin capas de costo por fecha de ingreso, no sería posible calcular PEPS ni conocer el costo real de cada venta.
Componentes standalone y signals en Angular	Es el enfoque recomendado en Angular 21 y evita la sobrecarga de `NgModule` y de la detección de cambios de `zone.js`.

12. Instalación para Desarrollo

Los pasos detallados de instalación y ejecución se documentan en el Manual de Usuario, sección 2. Para el desarrollo se recomienda además:

- Ejecutar las pruebas del backend con `./mvnw test` antes de generar el paquete final con `./mvnw clean package`.
- Verificar en cada arranque que la consola muestre `Successfully applied N migrations` y `Started BackendApplication` sin errores.
- No editar un archivo de migración Flyway que ya haya sido aplicado; cualquier cambio a la estructura debe realizarse en un nuevo archivo `V3__...sql`.
- Compilar el frontend para producción con `ng build`, que genera los archivos estáticos en `frontend/dist/`.

13. Conclusiones

El Mini ERP cumple con los requerimientos funcionales y técnicos establecidos: arquitectura cliente-servidor con aplicaciones independientes, base de datos relacional creada mediante migraciones versionadas, control de acceso por roles, consistencia del inventario respaldada por transacciones y bloqueos, aplicación verificable del método PEPS, generación de facturas y un módulo de reportes completo.

El diseño en capas, el uso de DTOs y la separación entre backend y frontend permiten que el sistema pueda mantenerse y ampliarse de forma ordenada, agregando nuevos módulos sin comprometer la lógica ya implementada.